

Podstawy Three.js

15 kwietnia 2026

1 Introduction

Celem jest wprowadzenie do podstaw biblioteki Three.js. Oto strona projektu Ricardo Cabello (autora biblioteki): <https://mrdoob.com/>. Oto strona biblioteki Three.js: <http://threejs.org>.

Three.js używa pojęć, które już znasz, takich jak obiekty geometryczne, transformacje, światła, materiały, tekstury i kamery. Ale ma też dodatkowe funkcje, które bazują na mocy i elastyczności WebGL.

Możesz pobrać three.js i przeczytać dokumentację na głównej stronie internetowej <http://threejs.org>. Pobieranie jest dość duże, ponieważ zawiera wiele przykładów i plików pomocy.

Podstawowe funkcje three.js są zdefiniowane w jednym dużym pliku JavaScript o nazwie "three.js", który można znaleźć w katalogu kompilacji w pliku do pobrania. Istnieje również mniejsza wersja "minified", three.min.js, która zawiera te same definicje w formacie, który nie jest czytelny dla człowieka.

Instalacja i import — zaktualizowane

Aby użyć three.js na stronie internetowej, należy dołączyć jeden z dwóch skryptów w elemencie `<script>` na stronie. Na przykład, zakładając, że three.min.js znajduje się w tym samym folderze, co strona internetowa, wtedy element skryptu będzie:

```
<script src="three.min.js"></script>
```

Uwaga! Powyższy sposób importu jest przestarzały. Od wersji r116 Three.js oficjalnie wspiera ES Modules. Zalecane podejście to instalacja przez npm i import modułowy:

```
npm install three  
  
import * as THREE from 'three';
```

Możliwe jest też użycie importmap bezpośrednio w HTML — przydatne na zajęciach bez npm:

```
<script type="importmap">  
  { "imports": { "three": "https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js",  
                  "three/addons/": "https://cdn.jsdelivr.net/npm/three@0.160.0/examples/jsm/" }  
}</script>  
<script type="module">  
  import * as THREE from 'three';  
</script>
```

Oprócz tego pobieranie core three.js zawiera katalog zawierający wiele przykładów i różne pliki obsługi, które są używane w przykładach. Chociaż będę trzymał się głównie core, skorzystam z kilku dodatków.

2 Scena, Renderer, kamera

Three.js działa z elementem HTML `<canvas>`, tym samym, co w przypadku grafiki 2D. W wielu przeglądarkach internetowych oprócz interfejsu API grafiki 2D, canvas obsługuje również rysowanie w 3D przy użyciu WebGL, który jest mniej więcej taki sam, jak może być z 2D API. WebGL jest zaimplementowany w Internet Explorerze 11, a także w najnowszych wersjach Chrome, Safari, Firefox i Edge. Działa również w przeglądarkach na wielu urządzeniach mobilnych.

Three.js to interfejs API zorientowanego obiektowo grafu sceny. Podstawową procedurą jest zbudowanie grafu sceny z obiektów three.js, a następnie renderowanie obrazu sceny, którą reprezentuje. Animację można zaimplementować, modyfikując właściwości grafu sceny między ramkami.

Biblioteka three.js składa się z dużej liczby klas. Trzy najbardziej podstawowe to `THREE.Scene`, `THREE.Camera` i `THREE.WebGLRenderer`. Program three.js będzie potrzebował co najmniej jednego obiektu każdego typu. Obiekty te są często przechowywane w zmiennych globalnych:

```
var scene, renderer, camera;
```

Zauważ, że prawie wszystkie trzy klasy i stałe, które będziemy używać, są właściwościami obiektu o nazwie `THREE`, a ich nazwy zaczynają się od "THREE". Czasem będę odnosił się do klas bez użycia tego prefiksu, ale prefiks musi zawsze być zawarty w rzeczywistym kodzie programu.

Obiekt `Scene` jest uchwytym dla wszystkich obiektów tworzących świat 3D, w tym światła, obiektów graficznych i prawdopodobnie kamer. Działa jako węzeł główny dla grafu sceny. Scenę można utworzyć jako obiekt bez parametrów:

```
scene = new THREE.Scene();
```

Istnieją dwa rodzaje kamer: jedna z projekcją ortograficzną, druga z projekcją perspektywiczną. Są reprezentowane przez klasy `THREE.OrthographicCamera` i `THREE.PerspectiveCamera`. Konstruktorzy określają projekcję, używając parametrów znanych z OpenGL:

```
camera = new THREE.OrthographicCamera(left, right, top, bottom, near, far);  
// lub  
camera = new THREE.PerspectiveCamera(fieldOfViewAngle, aspect, near, far);
```

Typowy kod do tworzenia kamery perspektywicznej to:

```
camera = new THREE.PerspectiveCamera(45, canvas.width / canvas.height, 1, 100);
```

THREE.WebGLRenderer — zmiany w r152+

Mechanizm renderujący, który renderuje za pomocą WebGL, jest instancją klasy `THREE.WebGLRenderer`. Jego konstruktor ma jeden

parametr, który jest obiektem JavaScript zawierającym ustawienia.
Przykład:

```
renderer = new THREE.WebGLRenderer({
  canvas: theCanvas,
  antialias: true
});
renderer.setPixelRatio(window.devicePixelRatio); // HiDPI

// Uwaga: gammaOutput (stare) zastąpione przez outputColorSpace (r152+)
renderer.outputColorSpace = THREE.SRGBColorSpace; // zamiast gammaOutput = true
```

Najważniejszą rzeczą, którą chcesz zrobić z rendererem, jest renderowanie obrazu. Do tego potrzebna jest także scena i kamera:

```
renderer.render(scena, kamera);
```

3 THREE.Object3D

Graf sceny three.js składa się z obiektów typu THREE.Object3D (w tym obiektów należących do podklas tej klasy). Kamery, światła i widoczne obiekty są reprezentowane przez podklasy Object3D. W rzeczywistości sama THREE.Scene jest również podklasą Object3D.

Każdy Obiekt3D zawiera listę obiektów podrzędnych, które są również typu Object3D. Metoda `node.add(obiekt)` dodaje obiekt do listy elementów podrzędnych węzła. Metoda `node.remove(object)` może być użyta do usunięcia obiektu z listy.

Obiekt3D, `obj`, ma powiązaną transformację, która jest podana przez właściwości `obj.scale`, `obj.rotation` i `obj.position`. Warto wiedzieć, że od r140 dostępne są również wygodne metody:

```
obj.translateX(1.0); // przesun w lokalnym układzie X
obj.rotateY(0.01);  // obróć wokół lokalnej osi Y
obj.lookAt(target); // skieruj w stronę punktu

// Grupowanie obiektów
const group = new THREE.Group();
group.add(meshA, meshB);
scene.add(group);
```

4 Obiekt, geometria, materiał

Widoczny obiekt w three.js składa się z punktów, linii lub trójkątów. Istnieje pięć klas reprezentujących te możliwości: THREE.Points dla punktów, THREE.Mesh dla trójkątów i trzy klasy dla linii: THREE.Line, THREE.LineSegments i THREE.LineLoop.

THREE.Geometry — USUNIĘTE (r125)

Uwaga! Klasa THREE.Geometry oraz właściwości `geometry.vertices` i `geometry.faces` zostały całkowicie usunięte w r125 (2021). Nie należy ich używać. Zamiast tego należy używać wyłącznie THREE.BufferGeometry i THREE.BufferAttribute:

```
// STARE – nie działa od r125:
var geometry = new THREE.Geometry();
geometry.vertices.push(new THREE.Vector3(0, 0, 0));
geometry.faces.push(new THREE.Face3(0, 1, 2));

// NOWE – THREE.BufferGeometry:
const geometry = new THREE.BufferGeometry();
const count = 1000;
const positions = new Float32Array(count * 3);
for (let i = 0; i < count; i++) {
    positions[i * 3] = 2 * Math.random() - 1;
    positions[i * 3 + 1] = 2 * Math.random() - 1;
    positions[i * 3 + 2] = 2 * Math.random() - 1;
}
geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
```

Wbudowane klasy geometrii (BoxGeometry, SphereGeometry, itp.) działają poprawnie i wewnętrznie używają BufferGeometry — nie wymagają zmian:

```
new THREE.BoxGeometry(1, 1, 1);
new THREE.SphereGeometry(1, 32, 16);
new THREE.CylinderGeometry(0.5, 0.5, 2, 16);
new THREE.PlaneGeometry(10, 10);
```

Materialy

Dla obiektów typu THREE.Points możemy użyć materiału typu THREE.PointsMaterial. Dla siatek zaleca się używać MeshStandardMaterial zamiast starszych MeshLambertMaterial / MeshPhongMaterial, ponieważ implementuje fizycznie poprawne renderowanie (PBR):

```
// Zalecane – PBR:
var mat = new THREE.MeshStandardMaterial({
    color: 0x8B4513,
    roughness: 0.8, // 0 = lustro, 1 = matowy
    metalness: 0.1, // 0 = plastik, 1 = metal
});

// Bez oświetlenia (bez zmian):
var basicMat = new THREE.MeshBasicMaterial({ color: 0xff0000 });

// Własny shader:
var shaderMat = new THREE.ShaderMaterial({
    vertexShader: vertGLSL,
    fragmentShader: fragGLSL,
    uniforms: { uTime: { value: 0 } },
});
```

Możliwe jest również przypisanie wartości do właściwości materiału po utworzeniu obiektu. Po uzyskaniu geometrii i materiału możemy użyć ich do utworzenia widocznego obiektu i dodania go do sceny:

```
var sphereOfPoints = new THREE.Points(geometry, pointMaterial);
scene.add(sphereOfPoints);
```

5 Światła

W porównaniu do geometrii i materiałów, światła są łatwe! Three.js ma kilka klas do reprezentowania światła. Klasy to podklasy `THREE.Object3D`. Przyjrzymy się światłom kierunkowym, światłom punktowym, światłom otoczenia i reflektorom.

Światło kierunkowe:

```
var light = new THREE.DirectionalLight(); // domyślnie białe
light.position.set(0, 0, 1);
scene.add(light);
```

Światło punktowe:

```
new THREE.PointLight(color, intensity, cutoff);
```

Światło otoczenia:

```
new THREE.AmbientLight(color);
```

Reflektor (spotlight):

```
new THREE.SpotLight(color, intensity, cutoff, coneAngle, exponent);
```

6 Pętla animacji

Animację można zaimplementować, modyfikując właściwości grafu sceny między ramkami przy użyciu `requestAnimationFrame`. Zalecane podejście to użycie `THREE.Timer` do uzyskania delta czasu, co zapewnia animację niezależną od liczby klatek na sekundę:

```
// Zalecane – THREE.Timer:
const clock = new THREE.Timer();

function animate() {
  requestAnimationFrame(animate);
  clock.update();
  const delta = clock.getDelta(); // czas od ostatniej klatki [s]
  const elapsed = clock.getElapsed();

  mesh.rotation.y += delta * 1.0; // 1 rad/s niezależnie od FPS
  shaderMat.uniforms.uTime.value = elapsed;

  renderer.render(scene, camera);
}
animate();

// Obsługa resize okna:
window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

7 Addons — OrbitControls, GLTFLoader

Wiele pomocniczych klas dostępnych jest w katalogu three/addons/ (dawniej three/examples/jsm/). Najważniejsze z nich to:

```
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

// OrbitControls – sterowanie kamerą myszą
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
// W pętli animacji wymagane: controls.update();

// GLTFLoader – ładowanie modeli 3D (standard branżowy)
const loader = new GLTFLoader();
loader.load('model.gltf', (gltf) => {
  scene.add(gltf.scene);
});
```

Literatura

Uwaga!

Tworzenie aplikacji z wykorzystaniem three.js:

<https://docplayer.pl/20001297-Pisanie-aplikacji-z-wykorzystaniem-three-js.html>

W języku angielskim:

- książka interakcyjna: Three.js: Basics
<http://math.hws.edu/graphicsbook/c5/s1.html> (rozdział 5.1)
- oficjalna dokumentacja i przykłady: <https://threejs.org/docs/> oraz <https://threejs.org/examples/>

8 Zadanie

Celem jest konstruowanie złożonego modelu za pomocą three.js - animowanej karuzeli (podstawa karuzeli jest wielokątem odpowiednio z konfiguracją zadania) i co najmniej jednego innego wybranego modelu. Pliki do pobrania znajdują się poniżej. Głównym plikiem jest lab9.html. Podfolder zasobów resources zawiera dwa pliki JavaScript używane przez program oraz model konia, którego używamy w karuzeli. Zawiera również kilka plików graficznych, które można wykorzystać jako tekstury.

Szkielet startowy (zaktualizowany do ES Modules):

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

const scene    = new THREE.Scene();
const camera   = new THREE.PerspectiveCamera(60, innerWidth/innerHeight, 0.1, 200);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
renderer.shadowMap.enabled = true;
document.body.appendChild(renderer.domElement);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
camera.position.set(0, 8, 18);

scene.add(new THREE.AmbientLight(0xffffff, 0.5));
const sun = new THREE.DirectionalLight(0xffffff, 1);
sun.position.set(10, 20, 10);
sun.castShadow = true;
scene.add(sun);

// --- buduj karuzelę tutaj ---

const clock = new THREE.Timer();
function animate() {
    requestAnimationFrame(animate);
    clock.update();
    const delta = clock.getDelta();
    controls.update();
    renderer.render(scene, camera);
}
animate();
```

Przykłady tworzenia modelu złożonego Three.js:

<http://math.hws.edu/graphicsbook/source/threejs/diskworld-1.html>

<http://math.hws.edu/eck/cs424/f17/lab9/windmill/windmill.html>